

6. File IO

Opening and closing files

- Open files using `open()`
- `open()` returns a file object, which handles the file, so it needs to be stored in a variable to access the file
- `open()` takes two parameters, the file name and the mode
- The file must exist in the same folder as your python notebook or file, or an error will be raised
- When the mode is not specified, it defaults to read mode

File mode	Action
"r"	Read
"w"	Write
"a"	Append

- Add a `b` at the end to the file mode to read binary data instead of text data
- Write creates a new file if it does not exist and writes to it, and overwrites over any existing data if the file exists and write data to it
- Append creates a new file if it does not exist and writes to it, and adds information to the end of the file if it exists

```
- text.txt -  
But when I stepped through,  
There was no floor.
```

```
file = open('text.txt', r)
```

- After opening the file we need to `close()` the while to release resources

```
file.close()
```

with method

- We can use a `with` block to automatically close a file after we are done

```
with open('text.txt', 'r') as file: #file will be closed after the with block  
    pass
```

- Our file here gets stored as the variable `file`
- We can do whatever we want inside of the with block
- After python is done executing the instructions inside the with block, it will automatically close the file
- We need to comment `#file will be closed after the with block` to get marks

Reading

read()

- `read()` reads the whole file

```
with open('text.txt', 'r') as file:
    text = file.read()

print(text)
print(repr(text))
```

```
But when I stepped through,
There was no floor.
'But when I stepped through,\nThere was no floor.'
```

- `repr()` shows escape characters such as newline characters, or tabs
- `read()` includes newline characters

readline()

```
with open('text.txt', 'r') as file:
    text = file.readline()
print(text)
print(repr(text))
```

```
But when I stepped through,

'But when I stepped through,\n'
```

- `readline()` reads the file line by line
- `readline()` includes newline characters
- Calling it again would move on to the next line

```
with open('text.txt', 'r') as file:
    for i in range(2):
        text = file.readline()
print(text)
print(repr(text))
```

```
There was no floor.
'There was no floor.'
```

- We can print out all of the lines as such

```
with open('text.txt', 'r') as file:
    while True:
        line = file.readline()
        if line: # if there are still lines
            print(line.strip())
        else: # we have reached the end of the file
            break
```

```
But when I stepped through,  
There was no floor.
```

readlines()

- `readlines()` reads out all the lines and stores it as an array

```
with open('text.txt', 'r') as file:  
    text = file.readlines()  
print(text)
```

```
['But when I stepped through,\n', 'There was no floor.']
```

Writing

- To write to a file, we need to open it in write mode or append mode

write()

- Write adds whatever we wrote to the file

```
with open('text.txt', 'w') as file:  
    file.write("You're coming back")  
    file.write("And it's the end of the world")  
    file.write("\nWe're starting over and I love you darling")  
  
with open('text.txt', 'r') as file:  
    text = file.read()  
print(text)  
  
print("")  
  
with open('text.txt', 'a') as file:  
    file.write("\nand I am done, dear")  
  
with open('text.txt', 'r') as file:  
    text = file.read()  
print(text)
```

```
You're coming backAnd it's the end of the world  
We're starting over and I love you darling
```

```
You're coming backAnd it's the end of the world  
We're starting over and I love you darling  
and I am done, dear
```

- We need to write `\n` when we want to start a new line
- We can also open the while in write mode to clear the file

```
with open('text.txt', 'w') as file:  
    pass  
  
with open('text.txt', 'r') as file:
```

```
text = file.read()
print(text)
```

writelines()

- Write lines write a list of strings into the target file

```
text = ["Hello", "world", "\nHi", "\nmom!"]
with open("text.txt", "w") as file:
    file.writelines(text)

with open("text.txt", "r") as file:
    text = file.read()

print(text)
```

```
Hello world
Hi
mom!
```

CSV module

- CSV files is a common text file format to store tabular data
- Each row container one record
- Records can have multiple attribute separated by commas
- Normally the first line is the header
- CSV is a common format for data exchange because it is simple can compact
- Most relational databases provide tools to import and export CSV files
- CSV files can be opened in excel
- Python has a CSV module for handling these files

csv.reader()

- The reader is a list of all of the files at once
- `next()` tells the reader to go to the next line

```
- data.csv -
Name, Age, Favourite Color, Height
neud, 17, Green, 178cm
ur mom, 45, Yellow, 160cm
```

```
import csv

with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for i in reader:
        print(i)
```

```
['Name', 'Age', 'Favourite Color', ' Height']
['neud', '17', 'Green', '178cm']
['ur mom', '45', 'Yellow', '160cm']
```

- Saving the result as a list

```
import csv

data = []
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for i in reader:
        data.append(i)

print(data)
```

```
[['Name', 'Age', 'Favourite Color', ' Height'], ['neud', '17', 'Green', '178cm'], ['ur mom', '45', 'Yellow', '160cm']]
```

- Saving the header and the main information as separate results

```
import csv

data = []
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    header = next(reader)
    for i in reader:
        data.append(i)

print(header)
print(data)
```

```
['Name', 'Age', 'Favourite Color', ' Height']
[['neud', '17', 'Green', '178cm'], ['ur mom', '45', 'Yellow', '160cm']]
```

csv.writer()

- `csv.writer()` can be used to write to a csv file
- `writerow()` can take a list or a tuple or a set, and write it as one row

```
import csv

with open("data.csv", "w") as file:
    writer = csv.writer(file)
    header = ['Name', 'Age', 'Favourite Color', ' Height']
    writer.writerow(header)

with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for i in reader:
        print(i)
```

```
['Name', 'Age', 'Favourite Color', ' Height']
```

- `writerows()` can take a nested list of data and write each nested list as one row

```
import csv

data = [['neud', '17', 'Green', '178cm'], ['ur mom', '45', 'Yellow', '160cm']]
with open("data.csv", "w") as file:
    writer = csv.writer(file)
    header = ['Name', 'Age', 'Favourite Color', ' Height']
    writer.writerow(header)
    writer.writerows(data)

with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for i in reader:
        print(i)
```

```
['Name', 'Age', 'Favourite Color', ' Height']
['neud', '17', 'Green', '178cm']
['ur mom', '45', 'Yellow', '160cm']
```

JSON module

- JSON is another popular syntax for storing and exchanging data
- Python also has a JSON module for handling these files
- These files are denoted as a dictionary

`dumps()` and `dump()`

- `dumps()` converts python data into json data
- It takes the data as the first parameter

```
import json

data = {"name": "neud", "age": 17}
jsonString = json.dumps(data)
print(jsonString)
```

```
{"name": "neud", "age": 17}
```

- `dump()` converts python data into json data, and saves it as a JSON file
- It takes data as the first parameter, and a file object as the second

`load()` and `loads()`

- `loads()` takes a JSON string and converts it into python data
- It takes the string as the first parameter

```
import json

data = '{"name": "neud", "age": 17}'
```

```
python = json.loads(data)
print(python)
print(type(python))
```

```
{'name': 'neud', 'age': 17}
<class 'dict'>
```

- `load()` takes a JSON file and converts it to python data

```
- user.json -
{"name": "neud", "age": 17}
```

```
import json

with open("user.json", "r") as file:
    data= json.load(file)
print(data)
```

```
{'name': 'neud', 'age': 17}
```